

# Taller Hadoop HDFS - Procesamiento de Alto Volumen de Datos

Juan Felipe Guzmán\*, Jesús Puenayan†, Julián Andrés Carrillo‡, Cristian Cardozo§,  
Jhonathan Murcia¶, David Castillo‡, Nicolás Gutiérrez‡

Pontificia Universidad Javeriana — Computación de Alto Desempeño, Bogotá, Colombia

\*Ing. Industrial †Ing. Electrónico ‡Ing. de Sistemas §Economista ¶Ing. de Software

**Resumen**—Este documento presenta, el desarrollo del taller práctico centrado en el Sistema de Archivos Distribuido Hadoop (HDFS). En su desarrollo, se categorizan y documentan los comandos principales del clúster mediante la ejecución de ejemplos prácticos.

**Index Terms**—Hadoop, HDFS, Sistemas Distribuidos, Big Data

## I. INTRODUCCIÓN

El Sistema de Archivos Distribuido Hadoop (HDFS) es un sistema de almacenamiento organizado bajo un modelo de maestro y esclavo. En este sentido, la arquitectura se clasifica en dos partes principales: un NameNode que controla el sistema, y múltiples DataNodes que almacenan físicamente los datos. Además, el sistema replica la información en varios equipos; por lo tanto, esta acción causa una alta tolerancia a los fallos técnicos impidiendo la pérdida de información.

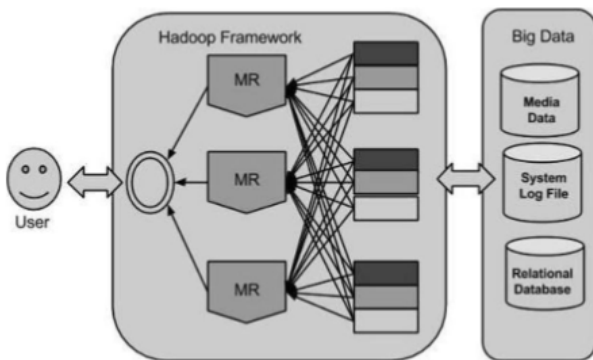


Figura 1. Arquitectura general del framework Hadoop: Interacción del usuario, procesamiento de tareas y almacenamiento en Big Data.

## II. OBJETIVOS

Los objetivos del taller son: practicar los comandos básicos y comprender la estructura del sistema.

## III. CONTEXTO Y PROPÓSITO DE LOS COMANDOS

El taller contiene múltiples comandos a ejecutar a continuación se describe el funcionamiento y propósito de cada uno para su posterior implementación:

- **Gestión de Usuarios:** El siguiente comando consiste en crear un grupo de multiusuarios llamado conda se le dan además permisos de usuario:

```
$ groupadd -g conda
$ sudo usermod -a -g conda usuario
```

- **Generación de Llaves de Autenticación:** Comando para generar una llave de autenticación para el nodo maestro

```
$ ssh-keygen
```

- **Distribución de Credenciales:** Comando para copiar la llave generada a todos los workers disponibles del cluster

```
$ ssh-copy-id workerA
```

- **Configuración del Sistema de Archivos:** Esta configuración indica que el sistema de archivos por defecto será HDFS, ubicado en el nodo maestro mediante la dirección hdfs://IPMASTER:9000 (IPMASTER = 10.43.97.147). Además, desactiva la validación estricta de permisos, define una ruta local para almacenar archivos temporales de Hadoop y habilita el uso de librerías nativas

```
<configuration>
  <property>
    <name>fs.defaultFS</name>
    <value>hdfs://IPMASTER:9000</value>
  </property>
  <property>
    <name>dfs.permissions</name>
    <value>>false</value>
  </property>
  <property>
    <name>hadoop.tmp.dir</name>
    <value>/mnt/sda1/hadoop/tmp</value>
  </property>
  <property>
    <name>hadoop.native.lib</name>
    <value>true</value>
  </property>
</configuration>
```

### III-A. Definición de Variables de Entorno

Esta configuración indica a Hadoop dónde se encuentra instalado Java, define la carpeta donde se guardarán los archivos de proceso del clúster y habilita el uso de librerías nativas de Hadoop

**Cuadro I**  
DESCRIPCIÓN Y PROPÓSITO DE LAS VARIABLES DE ENTORNO DE HADOOP.

| Propiedad                    | Propósito                                                                                                |
|------------------------------|----------------------------------------------------------------------------------------------------------|
| JAVA_HOME                    | Ruta de instalación de Java, necesario para ejecutar Hadoop.                                             |
| HADOOP_PID_DIR               | Indica dónde se almacenan los archivos PID de los procesos de Hadoop.                                    |
| HADOOP_OPTS                  | Agrega opciones generales para Hadoop, en este caso la ruta de las librerías nativas.                    |
| HADOOP_COMMON_LIB_NATIVE_DIR | Define la ubicación de las librerías nativas comunes de Hadoop.                                          |
| HDFS_NAMENODE_OPTS           | Configura opciones específicas del NameNode, usando ParallelGC y asignando un máximo de 4 GB de memoria. |

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_PID_DIR=/mnt/sda1/Cluster/pid
#Configuracion NameNode para usar parallelGC con 4GB Java Heap
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
export HADOOP_COMMON_LIB_NATIVE_DIR="$HADOOP_HOME/lib/native/"
export HDFS_NAMENODE_OPTS="-XX:+UseParallelGC -Xmx4g"
```

### III-B. Configuración de Propiedades HDFS

Esta configuración indica dónde se almacenará la información del NameNode y los bloques de datos de los DataNodes. También establece que cada bloque tendrá 2 réplicas, permitiendo tolerancia a fallos dentro del clúster.

**Cuadro II**  
DESCRIPCIÓN Y PROPÓSITO DE LAS PROPIEDADES DE CONFIGURACIÓN EN HDFS-SITE.XML.

| Propiedad               | Propósito                                                                                                                   |
|-------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| dfs.name.dir            | Define la ruta donde el NameNode almacenará sus metadatos, como la estructura de directorios y la ubicación de los bloques. |
| dfs.data.dir            | Define la ruta donde los DataNodes almacenarán físicamente los bloques de datos.                                            |
| dfs.replication         | Indica el número de copias que tendrá cada bloque. En este caso, cada bloque se replica 2 veces.                            |
| dfs.permission          | Desactiva el control de permisos de HDFS, útil en ambientes de práctica o laboratorio.                                      |
| dfs.webhdfs.enabled     | Habilita el acceso a HDFS mediante servicios web, permitiendo consultar o manipular archivos usando WebHDFS.                |
| ipc.maximum.data.length | Define el tamaño máximo de datos permitido en una comunicación IPC interna de Hadoop.                                       |

```
<configuration>
  <property>
    <name>dfs.name.dir</name>
    <value>/mnt/HadoopHDFS/NameNode</value>
  </property>
```

```
<property>
  <name>dfs.data.dir</name>
  <value>/mnt/HadoopHDFS/DataNode</value>
</property>
<property>
  <name>dfs.replication</name>
  <value>2</value>
</property>
<property>
  <name>dfs.permission</name>
  <value>>false</value>
</property>
<property>
  <name>dfs.webhdfs.enabled</name>
  <value>>true</value>
</property>
<property>
  <name>ipc.maximum.data.length</name>
  <value>134217728</value>
</property>
</configuration>
```

### III-B1. Configuración de rutas en el perfil de usuario:

Esta configuración define las rutas principales de instalación de Java y Hadoop, así como los directorios necesarios para ejecutar los componentes de Hadoop, HDFS, MapReduce y YARN. También agrega los comandos de Hadoop al PATH, permitiendo ejecutar instrucciones como hdfs, start-dfs.sh o stop-dfs.sh desde cualquier ubicación de la terminal

```
export JAVA_HOME=/usr/lib/jvm/java-8-openjdk-amd64
export HADOOP_HOME=/mnt/Cluster/Hadoop
export HADOOP_INSTALL=$HADOOP_HOME
export HADOOP_MAPRED_HOME=$HADOOP_HOME
export HADOOP_COMMON_HOME=$HADOOP_HOME
export HADOOP_HDFS_HOME=$HADOOP_HOME
export HADOOP_YARN_HOME=$HADOOP_HOME
export HADOOP_CONF_DIR=${HADOOP_HOME}/etc/hadoop
export HADOOP_COMMON_LIB_NATIVE_DIR=$HADOOP_HOME/lib/native
export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:$HADOOP_HOME/lib/native
export PATH=$PATH:$HADOOP_HOME/sbin:$HADOOP_HOME/bin
export HDFS_NAMENODE_USER="usuario"
export HDFS_DATANODE_USER="usuario"
export HDFS_SECONDARYNAMENODE_USER="usuario"
```

**III-B2. Creación de la carpeta NameNode en el nodo maestro:** Este comando se utiliza en el nodo maestro para crear la carpeta donde el NameNode almacenará la información administrativa o metadatos de HDFS.

```
$sudo mkdir -p /mnt/HadoopHDFS/NameNode

$chown -R :conda NameNode; chmod -R 777 NameNode
```

**III-B3. Creación de la carpeta DataNode en cada nodo worker:** Este comando se utiliza en cada nodo worker para crear la carpeta donde el DataNode almacenará físicamente los bloques de datos de HDFS

```
$sudo mkdir /mnt/HadoopHDFS/DataNode

$chown -R :conda DataNode; chmod -R 777
DataNode
```

## IV. DESARROLLO

Esta sección explica los pasos técnicos para instalar y configurar el clúster distribuido.

### IV-A. Configuración de Clúster Hadoop HDFS

Este apartado detalla el proceso inicial para preparar el entorno de trabajo. En este sentido, la configuración se clasifica en dos fases operativas: la descarga del software y la edición de las rutas.

**IV-A1. Descarga de la aplicación Hadoop:** El primer paso consiste en obtener el programa desde el sitio web. Se descarga la versión 3.5.0.

```
[estudiante@cad02-nfs01 almacen] $ curl https://d3ch0e.apache.org/hadoop/common/hadoop-3.5.0/hadoop-3.5.0.tar.gz
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 956048 100 956048    0     0  1511k      0  0 --:--:--  0 --:--:-- 1511k
Resolving d3ch0e.apache.org [d3ch0e.apache.org]: 151.101.2.132:80: non
Connecting to d3ch0e.apache.org [d3ch0e.apache.org]:151.101.2.132:80: connect
HTTP request sent, awaiting response... 200 OK
Length: 956048 (936K) [application/gzip]
Saving to: 'hadoop-3.5.0.tar.gz'
hadoop-3.5.0.tar.gz 100%[*****] 1511.35K 22.9MB/s   in 2.5s
```

Figura 2. Descarga de la versión 3.5.0 de Hadoop con el fin de configurar el clúster.

**IV-A2. Descompresión de HDFS:** El segundo paso consiste en la extracción y organización de los archivos del sistema. En este sentido, la tarea es descomprimir del paquete y el renombrar del directorio principal a "Hadoop".

```
[estudiante@cad02-nfs01 almacen] $ mv hadoop-3.5.0 Hadoop
[estudiante@cad02-nfs01 almacen] $ ll
total 956048
drwxr-xr-x 1 estudiante estudiante 4096 Mar 24 12:31 Hadoop
-rw-r--r-- 1 estudiante estudiante 581280793 Apr 1 15:43 hadoop-3.5.0.tar.gz
drwxr-xr-x 15 estudiante estudiante 4096 Apr 30 20:45 Spark
-rw-r--r-- 1 root root 401378872 Apr 30 19:14 spark-3.5.0-bin-hadoop3.tgz
[estudiante@cad02-nfs01 almacen] $
```

Figura 3. Descompresión de los binarios y renombramiento del directorio base a "Hadoop" en el almacenamiento del clúster.

**IV-A3. Edición del fichero de configuración principal:** El tercer paso consiste en modificar el archivo de configuración principal. En la edición se asigna de puertos y la definición de directorios.

```
lrwxrwxrwx. 1 root root 29 Feb 26 19:10 jre-openjdk -> /etc/alternatives/jre-openjdk
[estudiante@cad02-nfs01 jvm]$ cd /almacen/hadoop-3.5.0/etc/hadoop/
[estudiante@cad02-nfs01 hadoop]$ ll
total 188
-rw-r--r-- 1 estudiante estudiante 9213 Mar 24 12:14 capacity-scheduler.xml
-rw-r--r-- 1 estudiante estudiante 1335 Mar 24 12:15 configuration.xml
-rw-r--r-- 1 estudiante estudiante 2667 Mar 24 12:14 container-executor.cfg
-rw-r--r-- 1 estudiante estudiante 1093 May 14 16:55 core-site.xml
-rw-r--r-- 1 estudiante estudiante 3999 Mar 24 11:57 hadoop-env.cmd
-rw-r--r-- 1 estudiante estudiante 10798 May 14 19:02 hadoop-env.sh
-rw-r--r-- 1 estudiante estudiante 3321 Mar 24 11:57 hadoop-metrics2.properties
-rw-r--r-- 1 estudiante estudiante 14087 Mar 24 11:57 hadoop-policy.xml
-rw-r--r-- 1 estudiante estudiante 3414 Mar 24 11:57 hadoop-user-functions.sh.example
-rw-r--r-- 1 estudiante estudiante 683 Mar 24 12:07 hdfs-rbf-site.xml
-rw-r--r-- 1 estudiante estudiante 775 Mar 24 11:59 hdfs-site.xml
-rw-r--r-- 1 estudiante estudiante 1484 Mar 24 12:06 https-env.sh
-rw-r--r-- 1 estudiante estudiante 1657 Mar 24 12:06 https-log4j.properties
-rw-r--r-- 1 estudiante estudiante 620 Mar 24 12:08 https-site.xml
-rw-r--r-- 1 estudiante estudiante 3510 Mar 24 11:57 kms-acls.xml
-rw-r--r-- 1 estudiante estudiante 1351 Mar 24 11:57 kms-env.sh
-rw-r--r-- 1 estudiante estudiante 1771 Mar 24 11:57 kms-log4j.properties
-rw-r--r-- 1 estudiante estudiante 682 Mar 24 11:57 kms-site.xml
-rw-r--r-- 1 estudiante estudiante 10451 Mar 24 11:57 log4j.properties
-rw-r--r-- 1 estudiante estudiante 951 Mar 24 12:15 mapred-env.cmd
-rw-r--r-- 1 estudiante estudiante 1764 Mar 24 12:15 mapred-env.sh
-rw-r--r-- 1 estudiante estudiante 4113 Mar 24 12:15 mapred-queues.xml.template
-rw-r--r-- 1 estudiante estudiante 780 Mar 24 12:15 mapred-site.xml
drwxr-xr-x 2 estudiante estudiante 24 Mar 24 11:57 shellprofile.d
-rw-r--r-- 1 estudiante estudiante 2316 Mar 24 11:57 ssl-client.xml.example
-rw-r--r-- 1 estudiante estudiante 3766 Mar 24 11:57 ssl-server.xml.example
-rw-r--r-- 1 estudiante estudiante 2681 Mar 24 11:59 user-etc-policies.xml.template
-rw-r--r-- 1 estudiante estudiante 10 Mar 24 11:57 workers
-rw-r--r-- 1 estudiante estudiante 2250 Mar 24 12:14 yarn-env.cmd
-rw-r--r-- 1 estudiante estudiante 7895 Mar 24 12:14 yarn-env.sh
-rw-r--r-- 1 estudiante estudiante 2991 Mar 24 12:14 yarnservice-log4j.properties
-rw-r--r-- 1 estudiante estudiante 690 Mar 24 12:14 yarn-site.xml
[estudiante@cad02-nfs01 hadoop]$
```

Figura 4. Hadoop ubicado en los directorios de almacén correspondientes.

**IV-A4. Edición del fichero /Hadoop/etc/hadoop/hadoop-env.sh:** Este paso modifica del archivo que controla el entorno de ejecución. En este sentido, en las variables de configuración se indica la ubicación de Java y los parámetros de memoria del NameNode.

```
51 # The java implementation to use. By default, this environment
52 # variable is REQUIRED on ALL platforms except OS X!
53 # export JAVA_HOME=/usr/lib/jvm/java-17-openjdk-17.0.10-2.el9_7.x86_64
54 export HADOOP_PID_DIR=/almacen
55 export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
56 export HADOOP_COMMON_LIB_NATIVE_DIR="$HADOOP_HOME/lib/native/"
57 export HDFS_NAMENODE_OPTS="-XX:+UseParallelGC -Xmx4g"
58 # The language environment in which Hadoop runs. Use the English
59 # environment to ensure that logs are printed as expected.
60 export LANG=en_US.UTF-8
61
62 # Location of Hadoop. By default, Hadoop will attempt to determine
63 # this location based upon its execution path.
64 export HADOOP_HOME=
```

Figura 5. Edición de las variables de entorno de Java, almacenamiento y rendimiento en el archivo hadoop-env.sh.

```
# The java implementation to use. By default, this environment
# variable is REQUIRED on ALL platforms except OS X!
# export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-11.0.25.0-9-7.el9.x86_64
export HADOOP_PID_DIR=/almacen/Cluster/pid
export HADOOP_OPTS="-Djava.library.path=$HADOOP_HOME/lib/native"
export HADOOP_COMMON_LIB_NATIVE_DIR="$HADOOP_HOME/lib/native/"
export HDFS_NAMENODE_OPTS="-XX:+UseParallelGC -Xmx4g"
# The language environment in which Hadoop runs. Use the English
# environment to ensure that logs are printed as expected.
export LANG=en_US.UTF-8
```

Figura 6. Vista general y edición final del archivo script (.sh) dentro del editor de texto en la terminal.

**IV-A5. Se edita el fichero /Hadoop/etc/hadoop/hdfs-site.sh:**

**IV-A6. Edición del fichero /Hadoop/etc/hadoop/workers:** Este apartado describe el registro de las máquinas esclavas dentro de la configuración central del sistema. La información del clúster se asigna utilizando directamente los nombres de red de cada equipo en lugar de sus direcciones IP.

Las máquinas configuradas en el archivo se clasifican en el siguiente listado:

- cad02-nfs01
- cadclient02
- cadhead02
- cad02-w000
- cad02-w001
- cad02-w002
- cad02-w003
- cad02-w004





```

C:\Users\aulasingeneria>ssh estudiante@10.43.97.139
estudiante@10.43.97.139's password:
Last login: Thu May 14 20:16:03 2026 from 10.195.22.55
[estudiante@cadclient02 ~]$ cd /hadoop/
[estudiante@cadclient02 hadoop]$ mkdir DataNode
[estudiante@cadclient02 hadoop]$ ll
total 0
drwxr-xr-x 2 estudiante estudiante 6 May 14 20:17 DataNode
[estudiante@cadclient02 hadoop]$ chmod -R 777 DataNode/
[estudiante@cadclient02 hadoop]$ ll
total 0
drwxrwxrwx 2 estudiante estudiante 6 May 14 20:17 DataNode
[estudiante@cadclient02 hadoop]$ exit
logout
Connection to 10.43.97.139 closed.

C:\Users\aulasingeneria>ssh estudiante@10.43.97.140
estudiante@10.43.97.140's password:
Last login: Thu May 14 20:16:20 2026 from 10.195.22.55
[estudiante@cadhead02 ~]$ cd /hadoop
[estudiante@cadhead02 hadoop]$ mkdir DataNode
[estudiante@cadhead02 hadoop]$ chmod -R 777 DataNode/
[estudiante@cadhead02 hadoop]$ ll
total 0
drwxrwxrwx 2 estudiante estudiante 6 May 14 20:19 DataNode
[estudiante@cadhead02 hadoop]$ exit
logout
Connection to 10.43.97.140 closed.

```

Figura 11. Conexión mediante SSH a los nodos trabajadores para crear la carpeta DataNode y asignar los permisos totales (777).

#### IV-B. Comandos de Prueba

Este apartado describe el uso operativo de las instrucciones principales para gestionar el sistema de archivos distribuido. Bajo esta lógica, se ejecutan las sentencias básicas para explorar, crear y manipular la información directamente en el clúster. De esta manera, el uso de estos comandos causa la interacción física con el NameNode y los DataNodes; por lo tanto, permite validar el correcto funcionamiento de las operaciones de lectura y escritura. Sin embargo, omitir la sintaxis correcta generará errores de ejecución que impedirán la administración de los datos. En consecuencia, dominar estas instrucciones es obligatorio para operar el entorno de Hadoop.

**IV-B1. Comando `hdfs dfs -ls` /:** **Descripción:** Lista el contenido del directorio raíz en el sistema de archivos distribuido HDFS. Muestra los permisos, el número de réplicas de los bloques, el usuario/grupo propietario, el tamaño del archivo y la fecha de última modificación.

- **Ejemplo 1 (Listado del directorio raíz):**

```
$ hdfs dfs -ls /
```

**Análisis:** Este comando base permite comprobar que el NameNode formateado tiene la estructura inicial correcta y que el usuario estudiante tiene acceso al nivel más alto del sistema de ficheros.

- **Ejemplo 2 (Listado recursivo de un subdirectorio):**

```
$ hdfs dfs -ls -R /nombreA
```

**Análisis:** Usando la bandera `-R`, podemos listar todo el árbol de directorios y archivos que existan dentro de `/nombreA`, ideal para auditar la estructura de un proyecto complejo sin navegar carpeta por carpeta.

**IV-B2. Comando `hdfs dfs -mkdir /nombreA`:**

**Descripción:** Crea un nuevo directorio en la ruta especificada dentro del sistema distribuido. Funciona de manera idéntica al comando `mkdir` de Linux.

- **Ejemplo 1 (Creación de un directorio simple):**

```
$ hdfs dfs -mkdir /nombreA
$ hdfs dfs -mkdir /nombreB
```

**Análisis:** Genera un directorio de primer nivel para almacenar los datos.

- **Ejemplo 2 (Creación de árboles de directorios completos):**

```
$ hdfs dfs -mkdir -p /nombreA/experimentos
/ logs
```

**Análisis:** Al utilizar el parámetro `-p`, HDFS crea los directorios padres (experimentos) automáticamente si no existen. Esto previene errores de "directorio no encontrado." al desplegar scripts automatizados.

**IV-B3. Comando**

**`hdfs dfs -put dir_local/fichero /nombreA`:** **Descripción:** Copia uno o más archivos desde el sistema de archivos local del nodo (por ejemplo, el Master) hacia el sistema de archivos distribuido HDFS. Durante este proceso, el NameNode asigna los bloques y los DataNodes replican la información.

- **Ejemplo 1 (Subir un archivo específico):**

```
$ hdfs dfs -put /home/estudiante/Archivos-
prueba-hdfs/Prueba.txt /nombreA/
```

**Análisis:** Transfiere un archivo local pesado, como un modelo de ia o algo pequeño como un archivo txt (Prueba.txt) al clúster para que esté disponible para los workers.

- **Ejemplo 2 (Carga por lotes con comodines):**

```
$ hdfs dfs -put /home/estudiante/Archivos-
prueba-hdfs/*.txt /nombreA/
```

**Análisis:** Demuestra la capacidad de usar comodines (\*) para realizar una ingesta masiva de múltiples imágenes o archivos en una sola instrucción.

**IV-B4. Comando**

**`hdfs dfs -cat /nombreA/fichero`:** **Descripción:** Lee un archivo almacenado en HDFS y muestra su contenido directamente en la salida estándar (consola).

- **Ejemplo 1 (Lectura directa):**

```
$ hdfs dfs -cat /NombreA/Prueba.json
```

**Análisis:** Permite inspeccionar rápidamente archivos de texto o configuraciones JSON alojadas en el clúster sin necesidad de descargarlos previamente.

- **Ejemplo 2 (Procesamiento en tubería local):**

```
$ hdfs dfs -cat /nombreA/Prueba1.txt |
tail -n 20
```

**Análisis:** Al enviar la salida estándar a comandos nativos de Linux como `tail`, podemos visualizar solo las últimas 20 líneas de un registro que podría ser gigantesco, ahorrando memoria y tiempo en la consola.

IV-B5. Comando `hdfs dfs -get /nombreA/fichero dir_local/destino:`

**Descripción:** Realiza la operación inversa a `-put`. Descarga archivos o directorios completos desde HDFS hacia el disco duro local de la máquina donde se ejecuta el comando.

■ **Ejemplo 1 (Descarga de un archivo simple):**

```
$ hdfs dfs -get /nombreA/Prueba.txt /home/estudiante/Archivos-ingreso-hdfs/
```

**Análisis:** Permite extraer la salida de un proceso de alto rendimiento (como un CSV o un reporte) hacia el entorno local para su análisis final.

■ **Ejemplo 2 (Descarga forzada sobrescribiendo):**

```
$ hdfs dfs -get -f /nombreA/Prueba.txt /home/estudiante/Archivos-ingreso-hdfs
```

**Análisis:** El parámetro `-f` fuerza la descarga, sobrescribiendo cualquier archivo local que tenga el mismo nombre, lo cual es útil al actualizar constantemente un script local con la versión más reciente del clúster.

IV-B6. Comando `hdfs dfs -cp /nombreA/fichero /nombreB/`

**Descripción:** Copia archivos o directorios completos de un lugar a otro exclusivamente dentro del sistema de ficheros HDFS. No involucra tráfico hacia el sistema de archivos local.

■ **Ejemplo 1 (Copiar un archivo entre directorios):**

```
$ hdfs dfs -cp /nombreA/Prueba.txt /nombreB/
```

**Análisis:** Mueve utilidades o datos de un espacio de trabajo a otro sin requerir descargarlos y volverlos a subir, optimizando el ancho de banda de la red.

■ **Ejemplo 2 (Crear un respaldo interno de un directorio):**

```
$ hdfs dfs -cp /nombreA/Pruebal.txt /nombreB/backup
```

**Análisis:** Permite clonar un directorio completo dentro de HDFS antes de ejecutar un trabajo de procesamiento que modifique los datos, garantizando así un punto de restauración.

## V. CONCLUSIONES

- Se logró configurar correctamente el entorno de Hadoop HDFS para el almacenamiento distribuido de datos.
- La creación de directorios y la asignación de permisos permitieron preparar el clúster para recibir y administrar archivos.
- Los comandos básicos de HDFS facilitaron la carga, consulta, copia y descarga de archivos dentro del sistema distribuido.
- La prueba realizada confirmó que los datos fueron guardados y recuperados correctamente desde HDFS.
- El uso de réplicas en HDFS mejora la disponibilidad de la información y reduce el riesgo de pérdida de datos.

- En general, el taller permitió comprender de forma práctica cómo Hadoop administra archivos en un entorno distribuido.

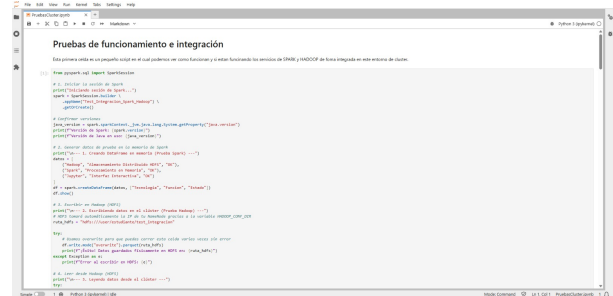


Figura 12. Ejecución exitosa de la integración entre Hadoop, Spark y Jupyter, mostrando el script de configuración y pruebas iniciales.



Figura 13. Resultados de la ejecución del script de integración, confirmando la lectura y escritura de DataFrames en HDFS.



Figura 14. Ejemplos prácticos de comandos HDFS en Jupyter: Comando 1 para la creación de directorios usando `mkdir`.

## REFERENCIAS

- [1] Dr. John Corredor, “Taller Hadoop HDFS: Procesamiento de Alto Volumen de Datos,” Departamento de Ingeniería de Sistemas, Pontificia Universidad Javeriana, Bogotá, Mayo 2026.
- [2] Apache Hadoop, “HDFS Architecture,” [En línea]. Disponible en: <https://hadoop.apache.org/docs/stable/hadoop-project-dist/hadoop-hdfs/HdfsDesign.html>

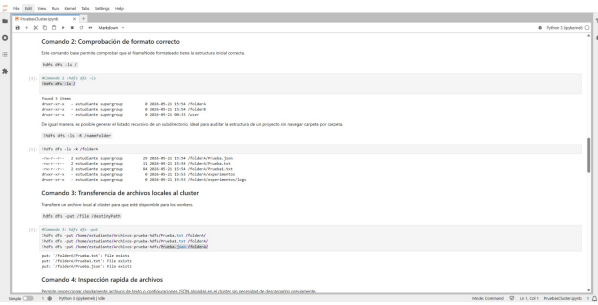


Figura 15. Ejemplos de comandos HDFS: Comando 2 para la comprobación de formato (ls) y Comando 3 para la transferencia de archivos locales al clúster (put).

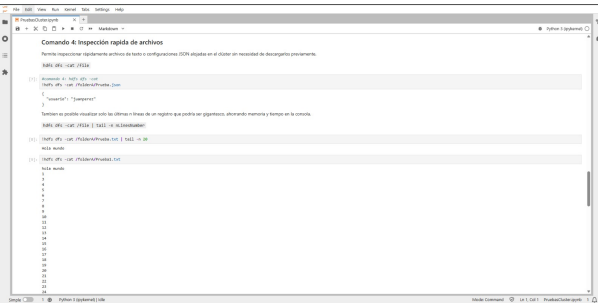


Figura 16. Ejemplos de comandos HDFS: Comando 4 para la inspección rápida de archivos utilizando cat y tail.

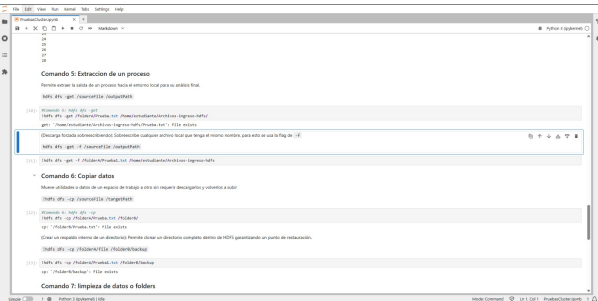


Figura 17. Ejemplos de comandos HDFS: Comando 5 para la extracción de procesos (get) y Comando 6 para copiar datos dentro de HDFS (cp).

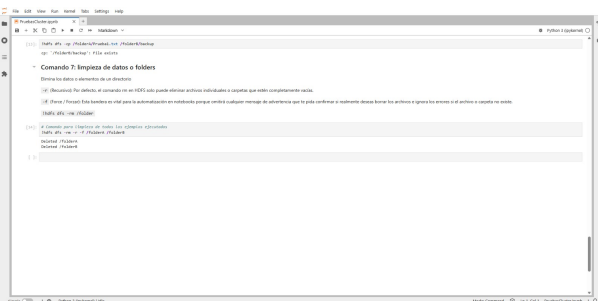


Figura 18. Ejemplos de comandos HDFS: Comando 7 para la limpieza de datos y eliminación de carpetas utilizando el comando rm.